Programming for AI
Week 5 Activity

Task 1: Sparse Vector Filtering & Index Mapping (List Comprehension)

Goal: In AI pipelines, feature vectors often contain numerous zero or near-zero values. Converting dense representations to sparse coordinate tuples avoids redundant computation.

The Problem: Write a function compress_sparse_vector(dense_vector, threshold=0.01) that accepts a 1D list of floating-point numbers.

Requirements:

- Filter out any values whose absolute magnitude |x| is less than or equal to threshold.
- Return a list of tuples in the format (index, value).
- **Constraint:** Implement the extraction using a single list comprehension without external accumulator loops or .append() calls.

Execution:

- Test input: `dense_vector = [0.0, 0.85, -0.002, 1.45, 0.0, -0.62, 0.009]`
- Expected output: `[(1, 0.85), (3, 1.45), (5, -0.62)]`

Task 2: Inverted Index & Token Frequency Mapping (Dictionary Comprehension)

Goal: Building vocabulary lookups and inverted frequency tables is a standard pre-processing step for natural language processing models.

The Problem: Write a function build_vocab_index(corpus) that accepts a list of raw document strings:

Requirements:

- Lowercase all incoming strings and split them into individual word tokens.
- Compute the total character count for each unique word across the collection.
- **Constraint:** Use dictionary comprehension paired with enumerate() or collection utilities to construct a dictionary where:
    - **Key:** The unique token string.
    - **Value:** A dictionary containing 'token_id' (its 0-based integer index in sorted alphabetical order) and 'char_len' (its length).

Execution:

- Test input: corpus = ["AI is transformative", "data is fuel for AI"]
- Verify that duplicate tokens (like "ai" and "is") map to a single entry and show their assigned identifiers.

Task 3: Outlier Clamping & Data Normalization (List Comprehension with If-Else)

Goal: When scaling data for neural network inputs, raw sensor signals frequently exceed normal bounds. If-else expressions within list comprehensions allow inline value transformation.

The Problem: Write a function clamp_and_label_signals(readings, lower_bound=-1.0, upper_bound=1.0):

Requirements:

- Implement a single list comprehension utilizing the ternary conditional syntax:
- [<expression_if_true> if <condition> else <expression_if_false> for item in iterable]
- For each value x:
    - If x > upper_bound, replace it with the string "UPPER_CLAMP".
    - If x < lower_bound, replace it with the string "LOWER_CLAMP".
    - Otherwise, round the float value to two decimal places.

Execution:

- Test input: `signals = [-2.5, -0.456, 0.123, 1.89, 0.999, -1.05]`
- Expected output: `['LOWER_CLAMP', -0.46, 0.12, 'UPPER_CLAMP', 1.0, 'LOWER_CLAMP']`

Task 4: Categorical Grade & Performance Discretization (Dictionary Comprehension with Conditional Branching)

Goal: Feature engineering often requires converting continuous scores into categorical labels for machine learning classification tasks.

The Problem: Given two synchronized lists:

student_ids = `["ST_101", "ST_102", "ST_103", "ST_104", "ST_105"]`

final_scores = `[92, 45, 78, 58, 81]`

Write a function `categorize_student_records(ids, scores):`

Requirements:

- Pair the lists using zip().
- Use a single dictionary comprehension with nested if-else branches:
    - Score >= 85 ==> "Distinction"
    - 60 <= Score < 85 ==> "Satisfactory"
    - Score < 60 ==> "Remedial"
- Return a dictionary where keys are student IDs and values are dictionaries with 'score' and 'tier'.

Execution:

- Validate that "ST_102" maps to "Remedial" and "ST_101" maps to "Distinction".

Task 5: Hyperparameter Sanitizer & Audit Logger (Function Wrappers)

Goal: In machine learning libraries, configuration functions must inspect, sanitize, and intercept arbitrary positional (*args) and keyword arguments (**kwargs) before forwarding them to underlying estimators.

The Problem: Implement an estimator constructor alongside an intercepting wrapper function:

Requirements:

- Define the target function init_model(model_type, learning_rate=0.01, epochs=100, **options). It should return a formatted dictionary confirming initialization settings.
- Define a wrapper function audit_wrapper(func, *args, **kwargs) that:
  - Unpacks all positional and keyword arguments using the splat operator (* and **).
  - Validates that learning_rate (if passed as an argument or keyword) satisfies 0.0 < learning_rate <= 1.0. If out of bounds, automatically clamp it to 0.01 and print a warning message.
  - Adds an audit timestamp entry {'audit_logged': True} into the keyword parameters before forwarding them to func.
  - Executes func with the sanitized parameters and returns its result.

Execution:

- Call `audit_wrapper(init_model, "GradientBoost", learning_rate=3.5, epochs=250, subsample=0.8)` and verify parameter interception and correction.

Task 6: Fault-Tolerant API Call Gatekeeper (Decorators)

Goal: Automated web scrapers and model inference APIs encounter intermittent network timeouts. A decorator can catch transient exceptions and retry requests automatically.

The Problem: Write a parameterized or standard decorator @retry_on_failure(max_retries=3, fallback_value=None):

Requirements:

- Wrap the decorated function inside a *args, **kwargs inner closure.
- Attempt execution within a loop up to max_retries times inside a try-except block.
- If the decorated function raises a ConnectionError or TimeoutError:
  - Print the failed attempt count.
  - Retry the call until attempts are exhausted.
- If all attempts fail, return fallback_value instead of crashing the program.

Application & Verification:

- Write an unreliable test function:

```
call_attempt = 0
def fetch_remote_weights():
    global call_attempt
    call_attempt += 1
    if call_attempt < 3:
        raise ConnectionError("Server reset connection.")
    return [0.12, 0.45, 0.89]
```

- Apply @retry_on_failure(max_retries=3, fallback_value=[]) and confirm recovery on the 3rd attempt.

Task 7: Computation Memoization & Model Inference Cache (Stateful Decorators)

Goal: Deep learning embedding lookups and intensive mathematical operations can be cached in memory to eliminate redundant calculations for recurring inputs.

The Problem: Write a decorator @memoize_predictions that adds memory persistence to a function:

Requirements:

- Maintain an internal cache storage dictionary within the enclosing scope of the decorator closure.
- The decorator must convert incoming positional arguments into a hashable cache key (e.g., converting a list argument to a tuple).
- **Gatekeeper Logic:**
    - If the input key exists in the cache, print "[CACHE HIT] Returning cached inference result" and immediately return the stored output without executing the wrapped function.
    - If not present, execute the function, store the result in the cache dictionary, print "[COMPUTE] Calculating fresh result", and return the newly computed value.

Execution:

- Apply @memoize_predictions to an expensive mathematical distance function:

```
def compute_feature_distance(vector_a, vector_b):
    return sum((x - y) ** 2 for x, y in zip(vector_a, vector_b)) ** 0.5
```

- Execute the decorated function twice with identical vector inputs: ([1, 2, 3], [4, 5, 6]). Demonstrate that the second execution skips computation and hits the cache.

Task 8: Memory Profile Comparison — Batch Processing vs. Lazy Generation (return vs. yield)

Goal: Understanding the exact mechanics of return (destroying local scope, loading full datasets into memory) versus yield (freezing execution state, streaming one element at a time) is essential for handling large-scale datasets.

The Problem: Write two contrasting functions that process continuous sensor readings:

Function A (generate_matrix_eager(n)):

- Uses standard for loops and return to build a full list of N synthetic feature vectors [i, i*0.5, i**2].

Function B (stream_matrix_lazy(n)):

- Uses yield inside a loop to output one feature vector at a time on demand.

Investigation & Deliverables:

- Import Python's sys module and compare memory consumption using sys.getsizeof() for both the eager list return object and the generator object when N = 1,000,000.
- Measure the initial execution delay before retrieving the very first element from each function.
- Write a brief markdown explanation answering:
    - *Why does the generator occupy fixed, negligible memory regardless of whether N=10 or N=10,000,000?*

Task 9: Sliding Window Sequence Sampler (Custom Iterator Protocol)

Goal: In time-series forecasting and sequential text models, input features are prepared as sliding contextual windows (e.g., predicting the next step based on the previous K values).

The Problem: Create a custom iterator class SlidingWindowIterator that complies strictly with Python's Iterator Protocol (__iter__ and __next__):

Class Architecture:

- __init__(self, sequence, window_size, step=1):
  - Stores the underlying data sequence (list), integer window_size, and movement step size.
  - Initializes an internal cursor index self.start_idx = 0.
- __iter__(self): Returns self.
- __next__(self):
  - Determines the slice boundary: self.start_idx to self.start_idx + self.window_size.
  - If the slice exceeds the total length of the sequence, raise StopIteration.
  - Advances self.start_idx by step and returns the window sub-list.

Execution:

- Test input: `timeseries = [10, 20, 30, 40, 50, 60, 70, 80], window_size = 3, step = 2`.
- Extract items using a standard for loop and verify the generated windows:
  - `[10, 20, 30]`
  - `[30, 40, 50]`
  - `[50, 60, 70]`

Task 10: Multi-Stage Lazy Image Preprocessing Pipeline (Chained Generators)

Goal: High-throughput vision data loaders chain multiple generator routines together so that resizing, augmentation, and normalization occur as a lazy dataflow without loading entire image folders into memory.

The Problem: Construct a three-stage generator pipeline using yield:

Stage 1 — Raw Streamer (stream_raw_images(image_records)):

- Accepts a list of mock dictionaries:

  `'image_id': int, 'pixels': list_of_ints_0_to_255}.`
- Yields each record one by one.

Stage 2 — Data Augmentor (augment_stream(generator_source)):

- Accepts the generator stream from Stage 1.
- Drops any record where the pixel list is empty.
- For valid records, yields the dictionary updated with a new key 'augmented': True.

Stage 3 — Min-Max Normalizer (normalize_stream(generator_source)):

- Accepts the generator stream from Stage 2.
- Transforms all raw pixel values [0, 255] into floating-point numbers in the range [0.0, 1.0] using:

  pixel_norm = pixel/255.0
- Yields the final processed record.

  Execution:

- Test dataset:
```
raw_batch = [
    {'image_id': 1, 'pixels': [0, 128, 255]},
    {'image_id': 2, 'pixels': []},
    {'image_id': 3, 'pixels': [64, 192, 32]}
]
```
- Chain the generators into a single processing line:
```
pipeline = normalize_stream(augment_stream(stream_raw_images(raw_batch)))
```
- Traverse the stream using a for loop, printing each fully processed record to prove lazy, end-to-end evaluation.